Annex B
(informative)
Library summary

B.1 Diagnostics <assert.h>

__STDC_VERSION_ASSERT_H__ **NDEBUG**

```
void assert(scalar expression);
```

B.2 Complex <complex.h>

__STDC_NO_COMPLEX__ **imaginary**
__STDC_VERSION_COMPLEX_H__ **_Imaginary_I**
complex **I**
_Complex_I

```
#pragma STDC CX_LIMITED_RANGE on-off-switch
double complex cacos(double complex z);
float complex cacosf(float complex z);
long double complex cacosl(long double complex z);
double complex casin(double complex z);
float complex casinf(float complex z);
long double complex casinl(long double complex z);
double complex catan(double complex z);
float complex catanf(float complex z);
long double complex catanl(long double complex z);
double complex ccos(double complex z);
float complex ccosf(float complex z);
long double complex ccosl(long double complex z);
double complex csin(double complex z);
float complex csinf(float complex z);
long double complex csinl(long double complex z);
double complex ctan(double complex z);
float complex ctanf(float complex z);
long double complex ctanl(long double complex z);
double complex cacosh(double complex z);
float complex cacoshf(float complex z);
long double complex cacoshl(long double complex z);
double complex casinh(double complex z);
float complex casinhf(float complex z);
long double complex casinhl(long double complex z);
double complex catanh(double complex z);
float complex catanhf(float complex z);
long double complex catanhl(long double complex z);
double complex ccosh(double complex z);
float complex ccoshf(float complex z);
long double complex ccoshl(long double complex z);
double complex csinh(double complex z);
float complex csinhf(float complex z);
long double complex csinhl(long double complex z);
double complex ctanh(double complex z);
float complex ctanhf(float complex z);
long double complex ctanhl(long double complex z);
double complex cexp(double complex z);
float complex cexpf(float complex z);
long double complex cexpl(long double complex z);
```


© ISO 2024 — All rights reserved

```
double complex clog(double complex z);
float complex clogf(float complex z);
long double complex clogl(long double complex z);
double cabs(double complex z);
float cabsf(float complex z);
long double cabsl(long double complex z);
double complex cpow(double complex x, double complex y);
float complex cpowf(float complex x, float complex y);
long double complex cpowl(long double complex x, long double complex y);
double complex csqrt(double complex z);
float complex csqrtf(float complex z);
long double complex csqrtl(long double complex z);
double carg(double complex z);
float cargf(float complex z);
long double cargl(long double complex z);
double cimag(double complex z);
float cimagf(float complex z);
long double cimagl(long double complex z);
double complex CMPLX(double x, double y);
float complex CMPLXF(float x, float y);
long double complex CMPLXL(long double complex x, long double y);
double complex conj(double complex z);
float complex conjf(float complex z);
long double complex conjl(long double complex z);
double complex cproj(double complex z);
float complex cprojf(float complex z);
long double complex cprojl(long double complex z);
double creal(double complex z);
float crealf(float complex z);
long double creall(long double complex z);
```

B.3 Character handling <ctype.h>

```
int isalnum(int c);
int isalpha(int c);
int isblank(int c);
int iscntrl(int c);
int isdigit(int c);
int isgraph(int c);
int islower(int c);
int isprint(int c);
int ispunct(int c);
int isspace(int c);
int isupper(int c);
int isxdigit(int c);
int tolower(int c);
int toupper(int c);
```

B.4 Errors <errno.h>

EDOM **EILSEQ** **ERANGE** **errno**

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of <errno.h>:

errno_t

B.5 Floating-point environment <fenv.h>

© ISO 2024 — All rights reserved

fenv_t
fexcept_t
femode_t
FE_DFL_MODE
FE_DIVBYZERO
FE_INEXACT
FE_INVALID
FE_OVERFLOW
FE_UNDERFLOW
FE_ALL_EXCEPT
FE_DOWNWARD
FE_TONEAREST
FE_TONEARESTFROMZERO
FE_TOWARDZERO
FE_UPWARD
FE_DFL_ENV
__STDC_VERSION_FENV_H__

```
#pragma STDC FENV_ACCESS on-off-switch
#pragma STDC FENV_ROUND direction
#pragma STDC FENV_ROUND FE_DYNAMIC
int feclearexcept(int excepts);
int fegetexceptflag(fexcept_t *flagp, int excepts);
int feraiseexcept(int excepts);
int fesetexcept(int excepts);
int fesetexceptflag(const fexcept_t *flagp, int excepts);
int fetestexceptflag(const fexcept_t *flagp, int excepts);
int fetestexcept(int excepts);
int fegetmode(femode_t *modep);
int fegetround(void);
int fesetmode(const femode_t *modep);
int fesetround(int rnd);
int fegetenv(fenv_t *envp);
int feholdexcept(fenv_t *envp);
int fesetenv(const fenv_t *envp);
int feupdateenv(const fenv_t *envp);
```

Only if the implementation defines **__STDC_IEC_60559_DFP__**:

FE_DEC_DOWNWARD
FE_DEC_TONEAREST
FE_DEC_TONEARESTFROMZERO
FE_DEC_TOWARDZERO
FE_DEC_UPWARD

```
#pragma STDC FENV_DEC_ROUND dec-direction
int fe_dec_getround(void);
int fe_dec_setround(int rnd);
```

Only if the implementation follows the recommended practice from F.2.2:

FE_SNANS_ALWAYS_SIGNAL

B.6 Characteristics of floating types <float.h>

B.6.1 Macros

__STDC_VERSION_FLOAT_H__
FLT_ROUNDS
FLT_EVAL_METHOD
FLT_HAS_SUBNORM
DBL_HAS_SUBNORM
LDBL_HAS_SUBNORM
FLT_RADIX
FLT_MANT_DIG
DBL_MANT_DIG
LDBL_MANT_DIG
FLT_DECIMAL_DIG
DBL_DECIMAL_DIG
LDBL_DECIMAL_DIG
DECIMAL_DIG
FLT_IS_IEC_60559
DBL_IS_IEC_60559
LDBL_IS_IEC_60559
FLT_DIG
DBL_DIG
LDBL_DIG
FLT_MIN_EXP
DBL_MIN_EXP
LDBL_MIN_EXP
FLT_MIN_10_EXP
DBL_MIN_10_EXP
LDBL_MIN_10_EXP
FLT_MAX_EXP
DBL_MAX_EXP
LDBL_MAX_EXP
FLT_MAX_10_EXP
DBL_MAX_10_EXP
LDBL_MAX_10_EXP
FLT_MAX
DBL_MAX
LDBL_MAX
FLT_NORM_MAX
DBL_NORM_MAX
LDBL_NORM_MAX
FLT_EPSILON

© ISO 2024 — All rights reserved

DBL_EPSILON	**FLT_SNAN**	**LDBL_TRUE_MIN**
LDBL_EPSILON	**DBL_SNAN**	**INFINITY**
FLT_MIN	**LDBL_SNAN**	**NAN**
DBL_MIN	**FLT_TRUE_MIN**	
LDBL_MIN	**DBL_TRUE_MIN**	

The following macro is provided only if the program defines **__STDC_WANT_IEC_60559_EXT__** before inclusion of the header `<float.h>`:

CR_DECIMAL_DIG

B.6.2 Characteristics of decimal floating types

1 The following macros are provided only if the implementation defines **__STDC_IEC_60559_DFP__**. *N* is 32, 64 and 128.

DEC_EVAL_METHOD	**DEC*N*_EPSILON**	**DEC*N*_MAX**	**DEC*N*_TRUE_MIN**
DEC_INFINITY	**DEC*N*_MANT_DIG**	**DEC*N*_MIN_EXP**	**DEC*N*_SNAN**
DEC_NAN	**DEC*N*_MAX_EXP**	**DEC*N*_MIN**	

B.7 Format conversion of integer types `<inttypes.h>`

__STDC_VERSION_INTTYPES_H__ **imaxdiv_t**

PRIb*N*	**PRIbLEAST*N***	**PRIbFAST*N***	**PRIbMAX**	**PRIbPTR**
PRIB*N*	**PRIBLEAST*N***	**PRIBFAST*N***	**PRIBMAX**	**PRIBPTR**
PRId*N*	**PRIdLEAST*N***	**PRIdFAST*N***	**PRIdMAX**	**PRIdPTR**
PRIi*N*	**PRIiLEAST*N***	**PRIiFAST*N***	**PRIiMAX**	**PRIiPTR**
PRIo*N*	**PRIoLEAST*N***	**PRIoFAST*N***	**PRIoMAX**	**PRIoPTR**
PRIu*N*	**PRIuLEAST*N***	**PRIuFAST*N***	**PRIuMAX**	**PRIuPTR**
PRIx*N*	**PRIxLEAST*N***	**PRIxFAST*N***	**PRIxMAX**	**PRIxPTR**
PRIX*N*	**PRIXLEAST*N***	**PRIXFAST*N***	**PRIXMAX**	**PRIXPTR**
SCNb*N*	**SCNbLEAST*N***	**SCNbFAST*N***	**SCNbMAX**	**SCNbPTR**
SCNd*N*	**SCNdLEAST*N***	**SCNdFAST*N***	**SCNdMAX**	**SCNdPTR**
SCNi*N*	**SCNiLEAST*N***	**SCNiFAST*N***	**SCNiMAX**	**SCNiPTR**
SCNo*N*	**SCNoLEAST*N***	**SCNoFAST*N***	**SCNoMAX**	**SCNoPTR**
SCNu*N*	**SCNuLEAST*N***	**SCNuFAST*N***	**SCNuMAX**	**SCNuPTR**
SCNx*N*	**SCNxLEAST*N***	**SCNxFAST*N***	**SCNxMAX**	**SCNxPTR**

```
intmax_t imaxabs(intmax_t j);
imaxdiv_t imaxdiv(intmax_t numer, intmax_t denom);
intmax_t strtoimax(const char * restrict nptr, char ** restrict endptr, int base);
uintmax_t strtoumax(const char * restrict nptr, char ** restrict endptr, int base);
intmax_t wcstoimax(const wchar_t *restrict nptr, wchar_t **restrict endptr, int base);
uintmax_t wcstoumax(const wchar_t *restrict nptr, wchar_t **restrict endptr, int base);
```

B.8 Alternative spellings `<iso646.h>`

and	**bitor**	**not_eq**	**xor**
and_eq	**compl**	**or**	**xor_eq**
bitand	**not**	**or_eq**	

B.9 Sizes of integer types `<limits.h>`

© ISO 2024 — All rights reserved

__STDC_VERSION_LIMITS_H__	ULONG_WIDTH	USHRT_MAX
BITINT_MAXWIDTH	LLONG_WIDTH	INT_MIN
BOOL_WIDTH	ULLONG_WIDTH	INT_MAX
CHAR_BIT	BOOL_MAX	UINT_MAX
CHAR_WIDTH	SCHAR_MIN	LONG_MIN
SCHAR_WIDTH	SCHAR_MAX	LONG_MAX
UCHAR_WIDTH	UCHAR_MAX	ULONG_MAX
SHRT_WIDTH	CHAR_MIN	LLONG_MIN
USHRT_WIDTH	CHAR_MAX	LLONG_MAX
INT_WIDTH	MB_LEN_MAX	ULLONG_MAX
UINT_WIDTH	SHRT_MIN	
LONG_WIDTH	SHRT_MAX	

B.10 Localization <locale.h>

struct lconv	LC_ALL	LC_CTYPE	LC_NUMERIC
NULL	LC_COLLATE	LC_MONETARY	LC_TIME

```
char *setlocale(int category, const char *locale);
struct lconv *localeconv(void);
```

B.11 Mathematics <math.h>

__STDC_VERSION_MATH_H__	FP_INT_TONEARESTFROMZERO	FP_FAST_DDIVL
float_t	FP_INT_TONEAREST	FP_FAST_FSQRT
double_t	FP_FAST_FMA	FP_FAST_FSQRTL
HUGE_VAL	FP_FAST_FMAF	FP_FAST_DSQRTL
HUGE_VALF	FP_FAST_FMAL	FP_FAST_FFMA
HUGE_VALL	FP_FAST_FADD	FP_FAST_FFMAL
INFINITY	FP_FAST_FADDL	FP_FAST_DFMAL
NAN	FP_FAST_DADDL	FP_ILOGB0
FP_INFINITE	FP_FAST_FSUB	FP_ILOGBNAN
FP_NAN	FP_FAST_FSUBL	FP_LLOGB0
FP_NORMAL	FP_FAST_DSUBL	FP_LLOGBNAN
FP_SUBNORMAL	FP_FAST_FMUL	MATH_ERRNO
FP_ZERO	FP_FAST_FMULL	MATH_ERREXCEPT
FP_INT_UPWARD	FP_FAST_DMULL	math_errhandling
FP_INT_DOWNWARD	FP_FAST_FDIV	
FP_INT_TOWARDZERO	FP_FAST_FDIVL	

```
#pragma STDC FP_CONTRACT on-off-switch
int fpclassify(real-floating x);
int iscanonical(real-floating x);
int isfinite(real-floating x);
int isinf(real-floating x);
int isnan(real-floating x);
int isnormal(real-floating x);
int signbit(real-floating x);
int issignaling(real-floating x);
int issubnormal(real-floating x);
int iszero(real-floating x);
double acos(double x);
float acosf(float x);
long double acosl(long double x);
double asin(double x);
float asinf(float x);
```


© ISO 2024 — All rights reserved

```
long double asinl(long double x);
double atan(double x);
float atanf(float x);
long double atanl(long double x);
double atan2(double y, double x);
float atan2f(float y, float x);
long double atan2l(long double y, long double x);
double cos(double x);
float cosf(float x);
long double cosl(long double x);
double sin(double x);
float sinf(float x);
long double sinl(long double x);
double tan(double x);
float tanf(float x);
long double tanl(long double x);
double acospi(double x);
float acospif(float x);
long double acospil(long double x);
double asinpi(double x);
float asinpif(float x);
long double asinpil(long double x);
double atanpi(double x);
float atanpif(float x);
long double atanpil(long double x);
double atan2pi(double y, double x);
float atan2pif(float y, float x);
long double atan2pil(long double y, long double x);
double cospi(double x);
float cospif(float x);
long double cospil(long double x);
double sinpi(double x);
float sinpif(float x);
long double sinpil(long double x);
double tanpi(double x);
float tanpif(float x);
long double tanpil(long double x);
double acosh(double x);
float acoshf(float x);
long double acoshl(long double x);
double asinh(double x);
float asinhf(float x);
long double asinhl(long double x);
double atanh(double x);
float atanhf(float x);
long double atanhl(long double x);
double cosh(double x);
float coshf(float x);
long double coshl(long double x);
double sinh(double x);
float sinhf(float x);
long double sinhl(long double x);
double tanh(double x);
float tanhf(float x);
long double tanhl(long double x);
double exp(double x);
float expf(float x);
long double expl(long double x);
double exp10(double x);
float exp10f(float x);
long double exp10l(long double x);
```

© ISO 2024 — All rights reserved

```
double exp10m1(double x);
float exp10m1f(float x);
long double exp10m1l(long double x);
double exp2(double x);
float exp2f(float x);
long double exp2l(long double x);
double exp2m1(double x);
float exp2m1f(float x);
long double exp2m1l(long double x);
double expm1(double x);
float expm1f(float x);
long double expm1l(long double x);
double frexp(double value, int *p);
float frexpf(float value, int *p);
long double frexpl(long double value, int *p);
int ilogb(double x);
int ilogbf(float x);
int ilogbl(long double x);
double ldexp(double x, int p);
float ldexpf(float x, int p);
long double ldexpl(long double x, int p);
long int llogb(double x);
long int llogbf(float x);
long int llogbl(long double x);
double log(double x);
float logf(float x);
long double logl(long double x);
double log10(double x);
float log10f(float x);
long double log10l(long double x);
double log10p1(double x);
float log10p1f(float x);
long double log10p1l(long double x);
double log1p(double x);
float log1pf(float x);
long double log1pl(long double x);
double logp1(double x);
float logp1f(float x);
long double logp1l(long double x);
double log2(double x);
float log2f(float x);
long double log2l(long double x);
double log2p1(double x);
float log2p1f(float x);
long double log2p1l(long double x);
double logb(double x);
float logbf(float x);
long double logbl(long double x);
double modf(double value, double *iptr);
float modff(float value, float *iptr);
long double modfl(long double value, long double *iptr);
double scalbn(double x, int n);
float scalbnf(float x, int n);
long double scalbnl(long double x, int n);
double scalbln(double x, long int n);
float scalblnf(float x, long int n);
long double scalblnl(long double x, long int n);
double cbrt(double x);
float cbrtf(float x);
long double cbrtl(long double x);
double compoundn(double x, long long int n);
```


© ISO 2024 — All rights reserved

```
float compoundnf(float x, long long int n);
long double compoundnl(long double x, long long int n);
double fabs(double x);
float fabsf(float x);
long double fabsl(long double x);
double hypot(double x, double y);
float hypotf(float x, float y);
long double hypotl(long double x, long double y);
double pow(double x, double y);
float powf(float x, float y);
long double powl(long double x, long double y);
double pown(double x, long long int n);
float pownf(float x, long long int n);
long double pownl(long double x, long long int n);
double powr(double y, double x);
float powrf(float y, float x);
long double powrl(long double y, long double x);
double rootn(double x, long long int n);
float rootnf(float x, long long int n);
long double rootnl(long double x, long long int n);
double rsqrt(double x);
float rsqrtf(float x);
long double rsqrtl(long double x);
double sqrt(double x);
float sqrtf(float x);
long double sqrtl(long double x);
double erf(double x);
float erff(float x);
long double erfl(long double x);
double erfc(double x);
float erfcf(float x);
long double erfcl(long double x);
double lgamma(double x);
float lgammaf(float x);
long double lgammal(long double x);
double tgamma(double x);
float tgammaf(float x);
long double tgammal(long double x);
double ceil(double x);
float ceilf(float x);
long double ceill(long double x);
double floor(double x);
float floorf(float x);
long double floorl(long double x);
double nearbyint(double x);
float nearbyintf(float x);
long double nearbyintl(long double x);
double rint(double x);
float rintf(float x);
long double rintl(long double x);
long int lrint(double x);
long int lrintf(float x);
long int lrintl(long double x);
long long int llrint(double x);
long long int llrintf(float x);
long long int llrintl(long double x);
double round(double x);
float roundf(float x);
long double roundl(long double x);
long int lround(double x);
long int lroundf(float x);
```

© ISO 2024 — All rights reserved

```
long int lroundl(long double x);
long long int llround(double x);
long long int llroundf(float x);
long long int llroundl(long double x);
double roundeven(double x);
float roundevenf(float x);
long double roundevenl(long double x);
double trunc(double x);
float truncf(float x);
long double truncl(long double x);
double fromfp(double x, int rnd, unsigned int width);
float fromfpf(float x, int rnd, unsigned int width);
long double fromfpl(long double x, int rnd, unsigned int width);
double ufromfp(double x, int rnd, unsigned int width);
float ufromfpf(float x, int rnd, unsigned int width);
long double ufromfpl(long double x, int rnd, unsigned int width);
double fromfpx(double x, int rnd, unsigned int width);
float fromfpxf(float x, int rnd, unsigned int width);
long double fromfpxl(long double x, int rnd, unsigned int width);
double ufromfpx(double x, int rnd, unsigned int width);
float ufromfpxf(float x, int rnd, unsigned int width);
long double ufromfpxl(long double x, int rnd, unsigned int width);
double fmod(double x, double y);
float fmodf(float x, float y);
long double fmodl(long double x, long double y);
double remainder(double x, double y);
float remainderf(float x, float y);
long double remainderl(long double x, long double y);
double remquo(double x, double y, int *quo);
float remquof(float x, float y, int *quo);
long double remquol(long double x, long double y, int *quo);
double copysign(double x, double y);
float copysignf(float x, float y);
long double copysignl(long double x, long double y);
double nan(const char *tagp);
float nanf(const char *tagp);
long double nanl(const char *tagp);
double nextafter(double x, double y);
float nextafterf(float x, float y);
long double nextafterl(long double x, long double y);
double nexttoward(double x, long double y);
float nexttowardf(float x, long double y);
long double nexttowardl(long double x, long double y);
double nextup(double x);
float nextupf(float x);
long double nextupl(long double x);
double nextdown(double x);
float nextdownf(float x);
long double nextdownl(long double x);
int canonicalize(double *cx, const double *x);
int canonicalizef(float *cx, const float *x);
int canonicalizel(long double *cx, const long double *x);
double fdim(double x, double y);
float fdimf(float x, float y);
long double fdiml(long double x, long double y);
double fmax(double x, double y);
float fmaxf(float x, float y);
long double fmaxl(long double x, long double y);
double fmin(double x, double y);
float fminf(float x, float y);
long double fminl(long double x, long double y);
```


© ISO 2024 — All rights reserved

```
double fmaximum(double x, double y);
float fmaximumf(float x, float y);
long double fmaximuml(long double x, long double y);
double fminimum(double x, double y);
float fminimumf(float x, float y);
long double fminimuml(long double x, long double y);
double fmaximum_mag(double x, double y);
float fmaximum_magf(float x, float y);
long double fmaximum_magl(long double x, long double y);
double fminimum_mag(double x, double y);
float fminimum_magf(float x, float y);
long double fminimum_magl(long double x, long double y);
double fmaximum_num(double x, double y);
float fmaximum_numf(float x, float y);
long double fmaximum_numl(long double x, long double y);
double fminimum_num(double x, double y);
float fminimum_numf(float x, float y);
long double fminimum_numl(long double x, long double y);
double fmaximum_mag_num(double x, double y);
float fmaximum_mag_numf(float x, float y);
long double fmaximum_mag_numl(long double x, long double y);
double fminimum_mag_num(double x, double y);
float fminimum_mag_numf(float x, float y);
long double fminimum_mag_numl(long double x, long double y);
double fma(double x, double y, double z);
float fmaf(float x, float y, float z);
long double fmal(long double x, long double y, long double z);
float fadd(double x, double y);
float faddl(long double x, long double y);
double daddl(long double x, long double y);
float fsub(double x, double y);
float fsubl(long double x, long double y);
double dsubl(long double x, long double y);
float fmul(double x, double y);
float fmull(long double x, long double y);
double dmull(long double x, long double y);
float fdiv(double x, double y);
float fdivl(long double x, long double y);
double ddivl(long double x, long double y);
float ffma(double x, double y, double z);
float ffmal(long double x, long double y, long double z);
double dfmal(long double x, long double y, long double z);
float fsqrt(double x);
float fsqrtl(long double x);
double dsqrtl(long double x);
int isgreater(real-floating x, real-floating y);
int isgreaterequal(real-floating x, real-floating y);
int isless(real-floating x, real-floating y);
int islessequal(real-floating x, real-floating y);
int islessgreater(real-floating x, real-floating y);
int isunordered(real-floating x, real-floating y);
int iseqsig(real-floating x, real-floating y);
```

Only if the implementation defines **__STDC_IEC_60559_DFP__**:

DEC_INFINITY
DEC_NAN
FP_FAST_FMAD32
FP_FAST_FMAD64
FP_FAST_FMAD128
FP_FAST_D32ADDD64
FP_FAST_D32ADDD128
FP_FAST_D64ADDD128
FP_FAST_D32SUBD64
FP_FAST_D32SUBD128
FP_FAST_D64SUBD128
FP_FAST_D32MULD64
FP_FAST_D32MULD128
FP_FAST_D64MULD128
FP_FAST_D32DIVD64

© ISO 2024 — All rights reserved

FP_FAST_D32DIVD128 **FP_FAST_D32FMAD128** **FP_FAST_D32SQRTD128**
FP_FAST_D64DIVD128 **FP_FAST_D64FMAD128** **FP_FAST_D64SQRTD128**
FP_FAST_D32FMAD64 **FP_FAST_D32SQRTD64**

```
_Decimal32 acosd32(_Decimal32 x);
_Decimal64 acosd64(_Decimal64 x);
_Decimal128 acosd128(_Decimal128 x);
_Decimal32 asind32(_Decimal32 x);
_Decimal64 asind64(_Decimal64 x);
_Decimal128 asind128(_Decimal128 x);
_Decimal32 atand32(_Decimal32 x);
_Decimal64 atand64(_Decimal64 x);
_Decimal128 atand128(_Decimal128 x);
_Decimal32 atan2d32(_Decimal32 y, _Decimal32 x);
_Decimal64 atan2d64(_Decimal64 y, _Decimal64 x);
_Decimal128 atan2d128(_Decimal128 y, _Decimal128 x);
_Decimal32 cosd32(_Decimal32 x);
_Decimal64 cosd64(_Decimal64 x);
_Decimal128 cosd128(_Decimal128 x);
_Decimal32 sind32(_Decimal32 x);
_Decimal64 sind64(_Decimal64 x);
_Decimal128 sind128(_Decimal128 x);
_Decimal32 tand32(_Decimal32 x);
_Decimal64 tand64(_Decimal64 x);
_Decimal128 tand128(_Decimal128 x);
_Decimal32 acospid32(_Decimal32 x);
_Decimal64 acospid64(_Decimal64 x);
_Decimal128 acospid128(_Decimal128 x);
_Decimal32 asinpid32(_Decimal32 x);
_Decimal64 asinpid64(_Decimal64 x);
_Decimal128 asinpid128(_Decimal128 x);
_Decimal32 atanpid32(_Decimal32 x);
_Decimal64 atanpid64(_Decimal64 x);
_Decimal128 atanpid128(_Decimal128 x);
_Decimal32 atan2pid32(_Decimal32 y, _Decimal32 x);
_Decimal64 atan2pid64(_Decimal64 y, _Decimal64 x);
_Decimal128 atan2pid128(_Decimal128 y, _Decimal128 x);
_Decimal32 cospid32(_Decimal32 x);
_Decimal64 cospid64(_Decimal64 x);
_Decimal128 cospid128(_Decimal128 x);
_Decimal32 sinpid32(_Decimal32 x);
_Decimal64 sinpid64(_Decimal64 x);
_Decimal128 sinpid128(_Decimal128 x);
_Decimal32 tanpid32(_Decimal32 x);
_Decimal64 tanpid64(_Decimal64 x);
_Decimal128 tanpid128(_Decimal128 x);
_Decimal32 acoshd32(_Decimal32 x);
_Decimal64 acoshd64(_Decimal64 x);
_Decimal128 acoshd128(_Decimal128 x);
_Decimal32 asinhd32(_Decimal32 x);
_Decimal64 asinhd64(_Decimal64 x);
_Decimal128 asinhd128(_Decimal128 x);
_Decimal32 atanhd32(_Decimal32 x);
_Decimal64 atanhd64(_Decimal64 x);
_Decimal128 atanhd128(_Decimal128 x);
_Decimal32 coshd32(_Decimal32 x);
_Decimal64 coshd64(_Decimal64 x);
_Decimal128 coshd128(_Decimal128 x);
_Decimal32 sinhd32(_Decimal32 x);
_Decimal64 sinhd64(_Decimal64 x);
```


© ISO 2024 — All rights reserved

```
_Decimal128 sinhd128(_Decimal128 x);
_Decimal32 tanhd32(_Decimal32 x);
_Decimal64 tanhd64(_Decimal64 x);
_Decimal128 tanhd128(_Decimal128 x);
_Decimal32 expd32(_Decimal32 x);
_Decimal64 expd64(_Decimal64 x);
_Decimal128 expd128(_Decimal128 x);
_Decimal32 exp10d32(_Decimal32 x);
_Decimal64 exp10d64(_Decimal64 x);
_Decimal128 exp10d128(_Decimal128 x);
_Decimal32 exp10m1d32(_Decimal32 x);
_Decimal64 exp10m1d64(_Decimal64 x);
_Decimal128 exp10m1d128(_Decimal128 x);
_Decimal32 exp2d32(_Decimal32 x);
_Decimal64 exp2d64(_Decimal64 x);
_Decimal128 exp2d128(_Decimal128 x);
_Decimal32 exp2m1d32(_Decimal32 x);
_Decimal64 exp2m1d64(_Decimal64 x);
_Decimal128 exp2m1d128(_Decimal128 x);
_Decimal32 expm1d32(_Decimal32 x);
_Decimal64 expm1d64(_Decimal64 x);
_Decimal128 expm1d128(_Decimal128 x);
_Decimal32 frexpd32(_Decimal32 value, int *p);
_Decimal64 frexpd64(_Decimal64 value, int *p);
_Decimal128 frexpd128(_Decimal128 value, int *p);
int ilogbd32(_Decimal32 x);
int ilogbd64(_Decimal64 x);
int ilogbd128(_Decimal128 x);
_Decimal32 ldexpd32(_Decimal32 x, int p);
_Decimal64 ldexpd64(_Decimal64 x, int p);
_Decimal128 ldexpd128(_Decimal128 x, int p);
long int llogbd32(_Decimal32 x);
long int llogbd64(_Decimal64 x);
long int llogbd128(_Decimal128 x);
_Decimal32 logd32(_Decimal32 x);
_Decimal64 logd64(_Decimal64 x);
_Decimal128 logd128(_Decimal128 x);
_Decimal32 log10d32(_Decimal32 x);
_Decimal64 log10d64(_Decimal64 x);
_Decimal128 log10d128(_Decimal128 x);
_Decimal32 log10p1d32(_Decimal32 x);
_Decimal64 log10p1d64(_Decimal64 x);
_Decimal128 log10p1d128(_Decimal128 x);
_Decimal32 log1pd32(_Decimal32 x);
_Decimal64 log1pd64(_Decimal64 x);
_Decimal128 log1pd128(_Decimal128 x);
_Decimal32 logp1d32(_Decimal32 x);
_Decimal64 logp1d64(_Decimal64 x);
_Decimal128 logp1d128(_Decimal128 x);
_Decimal32 log2d32(_Decimal32 x);
_Decimal64 log2d64(_Decimal64 x);
_Decimal128 log2d128(_Decimal128 x);
_Decimal32 log2p1d32(_Decimal32 x);
_Decimal64 log2p1d64(_Decimal64 x);
_Decimal128 log2p1d128(_Decimal128 x);
_Decimal32 logbd32(_Decimal32 x);
_Decimal64 logbd64(_Decimal64 x);
_Decimal128 logbd128(_Decimal128 x);
_Decimal32 modfd32(_Decimal32 x, _Decimal32 *iptr);
_Decimal64 modfd64(_Decimal64 x, _Decimal64 *iptr);
_Decimal128 modfd128(_Decimal128 x, _Decimal128 *iptr);
```

© ISO 2024 — All rights reserved

```
_Decimal32 scalbnd32(_Decimal32 x, int n);
_Decimal64 scalbnd64(_Decimal64 x, int n);
_Decimal128 scalbnd128(_Decimal128 x, int n);
_Decimal32 scalblnd32(_Decimal32 x, long int n);
_Decimal64 scalblnd64(_Decimal64 x, long int n);
_Decimal128 scalblnd128(_Decimal128 x, long int n);
_Decimal32 cbrtd32(_Decimal32 x);
_Decimal64 cbrtd64(_Decimal64 x);
_Decimal128 cbrtd128(_Decimal128 x);
_Decimal32 compoundnd32(_Decimal32 x, long long int n);
_Decimal64 compoundnd64(_Decimal64 x, long long int n);
_Decimal128 compoundnd128(_Decimal128 x, long long int n);
_Decimal32 fabsd32(_Decimal32 x);
_Decimal64 fabsd64(_Decimal64 x);
_Decimal128 fabsd128(_Decimal128 x);
_Decimal32 hypotd32(_Decimal32 x, _Decimal32 y);
_Decimal64 hypotd64(_Decimal64 x, _Decimal64 y);
_Decimal128 hypotd128(_Decimal128 x, _Decimal128 y);
_Decimal32 powd32(_Decimal32 x, _Decimal32 y);
_Decimal64 powd64(_Decimal64 x, _Decimal64 y);
_Decimal128 powd128(_Decimal128 x, _Decimal128 y);
_Decimal32 pownd32(_Decimal32 x, long long int n);
_Decimal64 pownd64(_Decimal64 x, long long int n);
_Decimal128 pownd128(_Decimal128 x, long long int n);
_Decimal32 powrd32(_Decimal32 y, _Decimal32 x);
_Decimal64 powrd64(_Decimal64 y, _Decimal64 x);
_Decimal128 powrd128(_Decimal128 y, _Decimal128 x);
_Decimal32 rootnd32(_Decimal32 x, long long int n);
_Decimal64 rootnd64(_Decimal64 x, long long int n);
_Decimal128 rootnd128(_Decimal128 x, long long int n);
_Decimal32 rsqrtd32(_Decimal32 x);
_Decimal64 rsqrtd64(_Decimal64 x);
_Decimal128 rsqrtd128(_Decimal128 x);
_Decimal32 sqrtd32(_Decimal32 x);
_Decimal64 sqrtd64(_Decimal64 x);
_Decimal128 sqrtd128(_Decimal128 x);
_Decimal32 erfd32(_Decimal32 x);
_Decimal64 erfd64(_Decimal64 x);
_Decimal128 erfd128(_Decimal128 x);
_Decimal32 erfcd32(_Decimal32 x);
_Decimal64 erfcd64(_Decimal64 x);
_Decimal128 erfcd128(_Decimal128 x);
_Decimal32 lgammad32(_Decimal32 x);
_Decimal64 lgammad64(_Decimal64 x);
_Decimal128 lgammad128(_Decimal128 x);
_Decimal32 tgammad32(_Decimal32 x);
_Decimal64 tgammad64(_Decimal64 x);
_Decimal128 tgammad128(_Decimal128 x);
_Decimal32 ceild32(_Decimal32 x);
_Decimal64 ceild64(_Decimal64 x);
_Decimal128 ceild128(_Decimal128 x);
_Decimal32 floord32(_Decimal32 x);
_Decimal64 floord64(_Decimal64 x);
_Decimal128 floord128(_Decimal128 x);
_Decimal32 nearbyintd32(_Decimal32 x);
_Decimal64 nearbyintd64(_Decimal64 x);
_Decimal128 nearbyintd128(_Decimal128 x);
_Decimal32 rintd32(_Decimal32 x);
_Decimal64 rintd64(_Decimal64 x);
_Decimal128 rintd128(_Decimal128 x);
long int lrintd32(_Decimal32 x);
```


© ISO 2024 — All rights reserved

```
long int lrintd64(_Decimal64 x);
long int lrintd128(_Decimal128 x);
long long int llrintd32(_Decimal32 x);
long long int llrintd64(_Decimal64 x);
long long int llrintd128(_Decimal128 x);
_Decimal32 roundd32(_Decimal32 x);
_Decimal64 roundd64(_Decimal64 x);
_Decimal128 roundd128(_Decimal128 x);
long int lroundd32(_Decimal32 x);
long int lroundd64(_Decimal64 x);
long int lroundd128(_Decimal128 x);
long long int llroundd32(_Decimal32 x);
long long int llroundd64(_Decimal64 x);
long long int llroundd128(_Decimal128 x);
_Decimal32 roundevend32(_Decimal32 x);
_Decimal64 roundevend64(_Decimal64 x);
_Decimal128 roundevend128(_Decimal128 x);
_Decimal32 truncd32(_Decimal32 x);
_Decimal64 truncd64(_Decimal64 x);
_Decimal128 truncd128(_Decimal128 x);
_Decimal32 fromfpd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 fromfpd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 fromfpd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 ufromfpd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 ufromfpd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 ufromfpd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 fromfpxd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 fromfpxd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 fromfpxd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 ufromfpxd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 ufromfpxd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 ufromfpxd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 fmodd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmodd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmodd128(_Decimal128 x, _Decimal128 y);
_Decimal32 remainderd32(_Decimal32 x, _Decimal32 y);
_Decimal64 remainderd64(_Decimal64 x, _Decimal64 y);
_Decimal128 remainderd128(_Decimal128 x, _Decimal128 y);
_Decimal32 copysignd32(_Decimal32 x, _Decimal32 y);
_Decimal64 copysignd64(_Decimal64 x, _Decimal64 y);
_Decimal128 copysignd128(_Decimal128 x, _Decimal128 y);
_Decimal32 nand32(const char *tagp);
_Decimal64 nand64(const char *tagp);
_Decimal128 nand128(const char *tagp);
_Decimal32 nextafterd32(_Decimal32 x, _Decimal32 y);
_Decimal64 nextafterd64(_Decimal64 x, _Decimal64 y);
_Decimal128 nextafterd128(_Decimal128 x, _Decimal128 y);
_Decimal32 nexttowardd32(_Decimal32 x, _Decimal128 y);
_Decimal64 nexttowardd64(_Decimal64 x, _Decimal128 y);
_Decimal128 nexttowardd128(_Decimal128 x, _Decimal128 y);
_Decimal32 nextupd32(_Decimal32 x);
_Decimal64 nextupd64(_Decimal64 x);
_Decimal128 nextupd128(_Decimal128 x);
_Decimal32 nextdownd32(_Decimal32 x);
_Decimal64 nextdownd64(_Decimal64 x);
_Decimal128 nextdownd128(_Decimal128 x);
int canonicalized32(_Decimal32 *cx, const _Decimal32 *x);
int canonicalized64(_Decimal64 *cx, const _Decimal64 *x);
int canonicalized128(_Decimal128 *cx, const _Decimal128 *x);
_Decimal32 fdimd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fdimd64(_Decimal64 x, _Decimal64 y);
```

© ISO 2024 — All rights reserved

```
_Decimal128 fdimd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaxd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaxd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaxd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmind32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmind64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmind128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaximumd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximumd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximumd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fminimumd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimumd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimumd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaximum_magd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_magd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_magd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fminimum_magd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_magd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_magd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaximum_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_numd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fminimum_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_numd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmaximum_mag_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_mag_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_mag_numd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fminimum_mag_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_mag_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_mag_numd128(_Decimal128 x, _Decimal128 y);
_Decimal32 fmad32(_Decimal32 x, _Decimal32 y, _Decimal32 z);
_Decimal64 fmad64(_Decimal64 x, _Decimal64 y, _Decimal64 z);
_Decimal128 fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
_Decimal32 d32addd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32addd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64addd128(_Decimal128 x, _Decimal128 y);
_Decimal32 d32subd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32subd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64subd128(_Decimal128 x, _Decimal128 y);
_Decimal32 d32muld64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32muld128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64muld128(_Decimal128 x, _Decimal128 y);
_Decimal32 d32divd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32divd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64divd128(_Decimal128 x, _Decimal128 y);
_Decimal32 d32fmad64(_Decimal64 x, _Decimal64 y, _Decimal64 z);
_Decimal32 d32fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
_Decimal64 d64fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
_Decimal32 d32sqrtd64(_Decimal64 x);
_Decimal32 d32sqrtd128(_Decimal128 x);
_Decimal64 d64sqrtd128(_Decimal128 x);
_Decimal32 quantized32(_Decimal32 x, _Decimal32 y);
_Decimal64 quantized64(_Decimal64 x, _Decimal64 y);
_Decimal128 quantized128(_Decimal128 x, _Decimal128 y);
bool samequantumd32(_Decimal32 x, _Decimal32 y);
bool samequantumd64(_Decimal64 x, _Decimal64 y);
bool samequantumd128(_Decimal128 x, _Decimal128 y);
_Decimal32 quantumd32(_Decimal32 x);
_Decimal64 quantumd64(_Decimal64 x);
_Decimal128 quantumd128(_Decimal128 x);
```


© ISO 2024 — All rights reserved

```
long long int llquantexpd32(_Decimal32 x);
long long int llquantexpd64(_Decimal64 x);
long long int llquantexpd128(_Decimal128 x);
void encodedecd32(unsigned char encptr[restrict static 4],
      const _Decimal32 * restrict xptr);
void encodedecd64(unsigned char encptr[restrict static 8],
      const _Decimal64 * restrict xptr);
void encodedecd128(unsigned char encptr[restrict static 16],
      const _Decimal128 * restrict xptr);
void decodedecd32(_Decimal32 * restrict xptr,
      const unsigned char encptr[restrict static 4]);
void decodedecd64(_Decimal64 * restrict xptr,
      const unsigned char encptr[restrict static 8]);
void decodedecd128(_Decimal128 * restrict xptr,
      const unsigned char encptr[restrict static 16]);
void encodebind32(unsigned char encptr[restrict static 4],
      const _Decimal32 * restrict xptr);
void encodebind64(unsigned char encptr[restrict static 8],
      const _Decimal64 * restrict xptr);
void encodebind128(unsigned char encptr[restrict static 16],
      const _Decimal128 * restrict xptr);
void decodebind32(_Decimal32 * restrict xptr,
      const unsigned char encptr[restrict static 4]);
void decodebind64(_Decimal64 * restrict xptr,
      const unsigned char encptr[restrict static 8]);
void decodebind128(_Decimal128 * restrict xptr,
      const unsigned char encptr[restrict static 16]);
```

Only if the implementation defines **__STDC_IEC_60559_BFP__** or **__STDC_IEC_559__** and additionally the user code defines **__STDC_WANT_IEC_60559_EXT__** before any inclusion of <math.h>:

```
int totalorder(const double *x, const double *y);
int totalorderf(const float *x, const float *y);
int totalorderl(const long double *x, const long double *y);
int totalordermag(const double *x, const double *y);
int totalordermagf(const float *x, const float *y);
int totalordermagl(const long double *x, const long double *y);
double getpayload(const double *x);
float getpayloadf(const float *x);
long double getpayloadl(const long double *x);
int setpayload(double *res, double pl);
int setpayloadf(float *res, float pl);
int setpayloadl(long double *res, long double pl);
int setpayloadsig(double *res, double pl);
int setpayloadsigf(float *res, float pl);
int setpayloadsigl(long double *res, long double pl);
```

Only if the implementation defines **__STDC_IEC_60559_DFP__** and additionally the user code defines **__STDC_WANT_IEC_60559_EXT__** before any inclusion of <math.h>:

_Decimal32_t **_Decimal64_t** **HUGE_VAL_D32** **HUGE_VAL_D64** **HUGE_VAL_D128**

```
int totalorderd32(const _Decimal32 *x, const _Decimal32 *y);
int totalorderd64(const _Decimal64 *x, const _Decimal64 *y);
int totalorderd128(const _Decimal128 *x, const _Decimal128 *y);
int totalordermagd32(const _Decimal32 *x, const _Decimal32 *y);
int totalordermagd64(const _Decimal64 *x, const _Decimal64 *y);
int totalordermagd128(const _Decimal128 *x, const _Decimal128 *y);
_Decimal32 getpayloadd32(const _Decimal32 *x);
_Decimal64 getpayloadd64(const _Decimal64 *x);
```

© ISO 2024 — All rights reserved

```
_Decimal128 getpayloadd128(const _Decimal128 *x);
int setpayloadd32(_Decimal32 *res, _Decimal32 pl);
int setpayloadd64(_Decimal64 *res, _Decimal64 pl);
int setpayloadd128(_Decimal128 *res, _Decimal128 pl);
int setpayloadsigd32(_Decimal32 *res, _Decimal32 pl);
int setpayloadsigd64(_Decimal64 *res, _Decimal64 pl);
int setpayloadsigd128(_Decimal128 *res, _Decimal128 pl);
```

B.12 Non-local jumps <setjmp.h>

__STDC_VERSION_SETJMP_H__ jmp_buf

```
int setjmp(jmp_buf env);
[[noreturn]] void longjmp(jmp_buf env, int val);
```

B.13 Signal handling <signal.h>

sig_atomic_t SIG_IGN SIGILL SIGTERM
SIG_DFL SIGABRT SIGINT
SIG_ERR SIGFPE SIGSEGV

```
void (*signal(int sig, void (*func)(int)))(int);
int raise(int sig);
```

B.14 Alignment <stdalign.h>

The header <stdalign.h> provides no content.

B.15 Variable arguments <stdarg.h>

va_list __STDC_VERSION_STDARG_H__

```
type va_arg(va_list ap, type);
void va_copy(va_list dest, va_list src);
void va_end(va_list ap);
void va_start(va_list ap, ...);
```

B.16 Atomics <stdatomic.h>

__STDC_NO_ATOMICS__
__STDC_VERSION_STDATOMIC_H__
ATOMIC_BOOL_LOCK_FREE
ATOMIC_CHAR_LOCK_FREE
ATOMIC_CHAR8_T_LOCK_FREE
ATOMIC_CHAR16_T_LOCK_FREE
ATOMIC_CHAR32_T_LOCK_FREE
ATOMIC_WCHAR_T_LOCK_FREE
ATOMIC_SHORT_LOCK_FREE
ATOMIC_INT_LOCK_FREE
ATOMIC_LONG_LOCK_FREE
ATOMIC_LLONG_LOCK_FREE
ATOMIC_POINTER_LOCK_FREE
ATOMIC_FLAG_INIT
memory_order
atomic_flag
memory_order_relaxed
memory_order_consume
memory_order_acquire
memory_order_release
memory_order_acq_rel
memory_order_seq_cst
atomic_bool
atomic_char
atomic_schar
atomic_uchar
atomic_short
atomic_ushort
atomic_int
atomic_uint

© ISO 2024 — All rights reserved

atomic_long
atomic_ulong
atomic_llong
atomic_ullong
atomic_char8_t
atomic_char16_t
atomic_char32_t
atomic_wchar_t
atomic_int_least8_t
atomic_uint_least8_t
atomic_int_least16_t
atomic_uint_least16_t
atomic_int_least32_t
atomic_uint_least32_t
atomic_int_least64_t
atomic_uint_least64_t
atomic_int_fast8_t
atomic_uint_fast8_t
atomic_int_fast16_t
atomic_uint_fast16_t
atomic_int_fast32_t
atomic_uint_fast32_t
atomic_int_fast64_t
atomic_uint_fast64_t
atomic_intptr_t
atomic_uintptr_t
atomic_size_t
atomic_ptrdiff_t
atomic_intmax_t
atomic_uintmax_t

```
void atomic_init(volatile A *obj, C value);
type kill_dependency(type y);
void atomic_thread_fence(memory_order order);
void atomic_signal_fence(memory_order order);
bool atomic_is_lock_free(const volatile A *obj);
void atomic_store(volatile A *object, C desired);
void atomic_store_explicit(volatile A *object, C desired, memory_order order);
C atomic_load(const volatile A *object);
C atomic_load_explicit(const volatile A *object, memory_order order);
C atomic_exchange(volatile A *object, C desired);
C atomic_exchange_explicit(volatile A *object, C desired, memory_order order);
bool atomic_compare_exchange_strong(volatile A *object, C *expected, C desired);
bool atomic_compare_exchange_strong_explicit(volatile A *object, C *expected,
      C desired, memory_order success, memory_order failure);
bool atomic_compare_exchange_weak(volatile A *object, C *expected, C desired);
bool atomic_compare_exchange_weak_explicit(volatile A *object, C *expected,
      C desired, memory_order success, memory_order failure);
C atomic_fetch_key(volatile A *object, M operand);
C atomic_fetch_key_explicit(volatile A *object, M operand, memory_order order);
bool atomic_flag_test_and_set(volatile atomic_flag *object);
bool atomic_flag_test_and_set_explicit(volatile atomic_flag *object,
      memory_order order);
void atomic_flag_clear(volatile atomic_flag *object);
void atomic_flag_clear_explicit(volatile atomic_flag *object,
      memory_order order);
```

B.17 Bit and byte utilities <stdbit.h>

__STDC_ENDIAN_BIG__
__STDC_ENDIAN_LITTLE__
__STDC_ENDIAN_NATIVE__
__STDC_VERSION_STDBIT_H__

```
unsigned int stdc_leading_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_leading_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_leading_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_leading_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_leading_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_leading_zeros(generic_value_type value) [[unsequenced]];
unsigned int stdc_leading_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_leading_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_leading_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_leading_ones_ul(unsigned long int value) [[unsequenced]];
```

© ISO 2024 — All rights reserved

```
unsigned int
stdc_leading_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_leading_ones(generic_value_type value) [[unsequenced]];
unsigned int stdc_trailing_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_trailing_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_trailing_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_trailing_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_trailing_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_trailing_zeros(generic_value_type value) [[unsequenced]];
unsigned int stdc_trailing_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_trailing_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_trailing_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_trailing_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_trailing_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_trailing_ones(generic_value_type value) [[unsequenced]];
unsigned int stdc_first_leading_zero_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_leading_zero_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_leading_zero_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_leading_zero_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_leading_zero_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_leading_zero(generic_value_type value) [[unsequenced]];
unsigned int stdc_first_leading_one_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_leading_one_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_leading_one_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_leading_one_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_leading_one_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_leading_one(generic_value_type value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_ui(unsigned int value) [[unsequenced]];
unsigned int
stdc_first_trailing_zero_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_trailing_zero_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_trailing_zero(generic_value_type value) [[unsequenced]];
unsigned int stdc_first_trailing_one_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_trailing_one_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_trailing_one_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_trailing_one_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_trailing_one_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_trailing_one(generic_value_type value) [[unsequenced]];
unsigned int stdc_count_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_count_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_count_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_count_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_count_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_count_zeros(generic_value_type value) [[unsequenced]];
unsigned int stdc_count_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_count_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_count_ones_ui(unsigned int value) [[unsequenced]];
```


© ISO 2024 — All rights reserved

```
unsigned int stdc_count_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_count_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_count_ones(generic_value_type value) [[unsequenced]];
bool stdc_has_single_bit_uc(unsigned char value) [[unsequenced]];
bool stdc_has_single_bit_us(unsigned short value) [[unsequenced]];
bool stdc_has_single_bit_ui(unsigned int value) [[unsequenced]];
bool stdc_has_single_bit_ul(unsigned long int value) [[unsequenced]];
bool stdc_has_single_bit_ull(unsigned long long int value) [[unsequenced]];
bool stdc_has_single_bit(generic_value_type value) [[unsequenced]];
unsigned int stdc_bit_width_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_bit_width_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_width_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_bit_width_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_bit_width_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_bit_width(generic_value_type value) [[unsequenced]];
unsigned char stdc_bit_floor_uc(unsigned char value) [[unsequenced]];
unsigned short stdc_bit_floor_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_floor_ui(unsigned int value) [[unsequenced]];
unsigned long int stdc_bit_floor_ul(unsigned long int value) [[unsequenced]];
unsigned long long int
stdc_bit_floor_ull(unsigned long long int value) [[unsequenced]];
generic_value_type stdc_bit_floor(generic_value_type value) [[unsequenced]];
unsigned char stdc_bit_ceil_uc(unsigned char value) [[unsequenced]];
unsigned short stdc_bit_ceil_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_ceil_ui(unsigned int value) [[unsequenced]];
unsigned long int stdc_bit_ceil_ul(unsigned long int value) [[unsequenced]];
unsigned long long int
stdc_bit_ceil_ull(unsigned long long int value) [[unsequenced]];
generic_value_type stdc_bit_ceil(generic_value_type value) [[unsequenced]];
```

B.18 Boolean type and values <stdbool.h>

__bool_true_false_are_defined

B.19 Checked Integer Operations <stdckdint.h>

__STDC_VERSION_STDCKDINT_H__

```
bool ckd_add(type1 *result, type2 a, type3 b);
bool ckd_sub(type1 *result, type2 a, type3 b);
bool ckd_mul(type1 *result, type2 a, type3 b);
```

B.20 Common definitions <stddef.h>

ptrdiff_t **max_align_t** **NULL**
nullptr_t **wchar_t**
size_t **__STDC_VERSION_STDDEF_H__**

```
      offsetof(type, member-designator)
```

```
      unreachable()
```

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of <stddef.h>:

© ISO 2024 — All rights reserved

rsize_t

B.21 Integer types <stdint.h>

int_N_**_t**
uint_N_**_t**
int_least_N_**_t**
uint_least_N_**_t**
int_fast_N_**_t**
uint_fast_N_**_t**
intptr_t
uintptr_t
intmax_t
uintmax_t
__STDC_VERSION_STDINT_H__
INT_N_**_MIN**
INT_N_**_MAX**
INT_N_**_WIDTH**
UINT_N_**_MAX**
UINT_N_**_WIDTH**
INT_LEAST_N_**_MIN**
INT_LEAST_N_**_MAX**
INT_LEAST_N_**_WIDTH**
UINT_LEAST_N_**_MAX**
UINT_LEAST_N_**_WIDTH**
INT_FAST_N_**_MIN**
INT_FAST_N_**_MAX**
INT_FAST_N_**_WIDTH**
UINT_FAST_N_**_MAX**
UINT_FAST_N_**_WIDTH**
INTPTR_MIN
INTPTR_MAX
INTPTR_WIDTH
UINTPTR_MAX
UINTPTR_WIDTH
INTMAX_MIN
INTMAX_MAX
INTMAX_WIDTH
UINTMAX_MAX
UINTMAX_WIDTH
PTRDIFF_MIN
PTRDIFF_MAX
SIG_ATOMIC_MIN
SIG_ATOMIC_MAX
SIG_ATOMIC_WIDTH
SIZE_MAX
SIZE_WIDTH
WCHAR_MIN
WCHAR_MAX
WCHAR_WIDTH
WINT_MIN
WINT_MAX
WINT_WIDTH
INT_N_**_C**(*value*)
UINT_N_**_C**(*value*)
INTMAX_C(*value*)
UINTMAX_C(*value*)

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of <stdint.h>:

RSIZE_MAX

B.22 Input/output <stdio.h>

size_t
FILE
fpos_t
NULL
_IOFBF
_IOLBF
_IONBF
BUFSIZ
EOF
FOPEN_MAX
FILENAME_MAX
L_tmpnam
SEEK_CUR
SEEK_END
SEEK_SET
TMP_MAX
stderr
stdin
stdout
_PRINTF_NAN_LEN_MAX
__STDC_VERSION_STDIO_H__

```
int remove(const char *filename);
int rename(const char *old, const char *new);
FILE *tmpfile(void);
char *tmpnam(char *s);
int fclose(FILE *stream);
int fflush(FILE *stream);
FILE *fopen(const char * restrict filename, const char * restrict mode);
FILE *freopen(const char * restrict filename, const char * restrict mode,
      FILE * restrict stream);
void setbuf(FILE * restrict stream, char * restrict buf);
int setvbuf(FILE * restrict stream, char * restrict buf, int mode, size_t size);
int printf(const char * restrict format, ...);
int scanf(const char * restrict format, ...);
int snprintf(char * restrict s, size_t n, const char * restrict format, ...);
int sprintf(char * restrict s, const char * restrict format, ...);
int sscanf(const char * restrict s, const char * restrict format, ...);
int vfprintf(FILE * restrict stream, const char * restrict format, va_list arg);
int vfscanf(FILE * restrict stream, const char * restrict format, va_list arg);
int vprintf(const char * restrict format, va_list arg);
```


© ISO 2024 — All rights reserved

```
int vscanf(const char * restrict format, va_list arg);
int vsnprintf(char * restrict s, size_t n, const char * restrict format, va_list arg);
int vsprintf(char * restrict s, const char * restrict format, va_list arg);
int vsscanf(const char * restrict s, const char * restrict format, va_list arg);
int fgetc(FILE *stream);
char *fgets(char * restrict s, int n, FILE * restrict stream);
int fputc(int c, FILE *stream);
int fputs(const char * restrict s, FILE * restrict stream);
int getc(FILE *stream);
int getchar(void);
int putc(int c, FILE *stream);
int putchar(int c);
int puts(const char *s);
int ungetc(int c, FILE *stream);
size_t fread(void * restrict ptr, size_t size, size_t nmemb,
      FILE * restrict stream);
size_t fwrite(const void * restrict ptr, size_t size, size_t nmemb,
      FILE * restrict stream);
int fgetpos(FILE * restrict stream, fpos_t * restrict pos);
int fseek(FILE *stream, long int offset, int whence);
int fsetpos(FILE *stream, const fpos_t *pos);
long int ftell(FILE *stream);
void rewind(FILE *stream);
void clearerr(FILE *stream);
int feof(FILE *stream);
int ferror(FILE *stream);
void perror(const char *s);
int fprintf(FILE * restrict stream, const char * restrict format, ...);
int fscanf(FILE * restrict stream, const char * restrict format, ...);
```

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of <stdio.h>:

L_tmpnam_s **TMP_MAX_S** **errno_t** **rsize_t**

```
errno_t tmpfile_s(FILE * restrict * restrict streamptr);
errno_t tmpnam_s(char *s, rsize_t maxsize);
errno_t fopen_s(FILE * restrict * restrict streamptr,
      const char * restrict filename, const char * restrict mode);
errno_t freopen_s(FILE * restrict * restrict newstreamptr,
      const char * restrict filename, const char * restrict mode,
      FILE * restrict stream);
int fprintf_s(FILE * restrict stream, const char * restrict format, ...);
int fscanf_s(FILE * restrict stream, const char * restrict format, ...);
int printf_s(const char * restrict format, ...);
int scanf_s(const char * restrict format, ...);
int snprintf_s(char * restrict s, rsize_t n, const char * restrict format, ...);
int sprintf_s(char * restrict s, rsize_t n, const char * restrict format, ...);
int sscanf_s(const char * restrict s, const char * restrict format, ...);
int vfprintf_s(FILE * restrict stream, const char * restrict format, va_list arg);
int vfscanf_s(FILE * restrict stream, const char * restrict format, va_list arg);
int vprintf_s(const char * restrict format, va_list arg);
int vscanf_s(const char * restrict format, va_list arg);
int vsnprintf_s(char * restrict s, rsize_t n, const char * restrict format,
      va_list arg);
int vsprintf_s(char * restrict s, rsize_t n, const char * restrict format,
      va_list arg);
int vsscanf_s(const char * restrict s, const char * restrict format, va_list arg);
char *gets_s(char *s, rsize_t n);
```

© ISO 2024 — All rights reserved

B.23 General utilities <stdlib.h>

size_t	**once_flag**	**NULL**
wchar_t	**__STDC_VERSION_STDLIB_H__**	**ONCE_FLAG_INIT**
div_t	**EXIT_FAILURE**	**RAND_MAX**
ldiv_t	**EXIT_SUCCESS**	
lldiv_t	**MB_CUR_MAX**	

```
void call_once(once_flag *flag, void (*func)(void));
double atof(const char *nptr);
int atoi(const char *nptr);
long int atol(const char *nptr);
long long int atoll(const char *nptr);
int strfromd(char * restrict s, size_t n, const char * restrict format,
    double fp);
int strfromf(char * restrict s, size_t n, const char * restrict format,
    float fp);
int strfroml(char * restrict s, size_t n, const char * restrict format,
    long double fp);
double strtod(const char * restrict nptr, char ** restrict endptr);
float strtof(const char * restrict nptr, char ** restrict endptr);
long double strtold(const char * restrict nptr, char ** restrict endptr);
long int strtol(const char * restrict nptr, char ** restrict endptr, int base);
long long int strtoll(const char * restrict nptr, char ** restrict endptr,
    int base);
unsigned long int strtoul(const char * restrict nptr, char ** restrict endptr,
    int base);
unsigned long long int strtoull(const char * restrict nptr,
    char ** restrict endptr, int base);
int rand(void);
void srand(unsigned int seed);
void *aligned_alloc(size_t alignment, size_t size);
void *calloc(size_t nmemb, size_t size);
void free(void *ptr);
void free_sized(void *ptr, size_t size);
void free_aligned_sized(void *ptr, size_t alignment, size_t size);
void *malloc(size_t size);
void *realloc(void *ptr, size_t size);
[[noreturn]] void abort(void);
int atexit(void (*func)(void));
int at_quick_exit(void (*func)(void));
[[noreturn]] void exit(int status);
[[noreturn]] void _Exit(int status);
char *getenv(const char *name);
[[noreturn]] void quick_exit(int status);
int system(const char *string);
QVoid *bsearch(const void *key, QVoid *base, size_t nmemb, size_t size,
      int (*compar)(const void *, const void *));
void qsort(void *base, size_t nmemb, size_t size,
      int (*compar)(const void *, const void *));
int abs(int j);
long int labs(long int j);
long long int llabs(long long int j);
div_t div(int numer, int denom);
ldiv_t ldiv(long int numer, long int denom);
lldiv_t lldiv(long long int numer, long long int denom);
int mblen(const char *s, size_t n);
int mbtowc(wchar_t * restrict pwc, const char * restrict s, size_t n);
int wctomb(char *s, wchar_t wc);
size_t mbstowcs(wchar_t * restrict pwcs, const char * restrict s, size_t n);
```


© ISO 2024 — All rights reserved

```
size_t wcstombs(char * restrict s, const wchar_t * restrict pwcs, size_t n);
size_t memalignment(const void *p);
```

Only if the implementation defines **__STDC_IEC_60559_DFP__**:

```
int strfromd32(char * restrict s, size_t n, const char * restrict format,
    _Decimal32 fp);
int strfromd64(char * restrict s, size_t n, const char * restrict format,
    _Decimal64 fp);
int strfromd128(char * restrict s, size_t n, const char * restrict format,
    _Decimal128 fp);
_Decimal32 strtod32(const char * restrict nptr, char ** restrict endptr);
_Decimal64 strtod64(const char * restrict nptr, char ** restrict endptr);
_Decimal128 strtod128(const char * restrict nptr, char ** restrict endptr);
```

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of `<stdlib.h>`:

errno_t **rsize_t** **constraint_handler_t**

```
constraint_handler_t set_constraint_handler_s(constraint_handler_t handler);
void abort_handler_s(const char * restrict msg, void * restrict ptr,
      errno_t error);
void ignore_handler_s(const char * restrict msg, void * restrict ptr,
      errno_t error);
errno_t getenv_s(size_t * restrict len, char * restrict value, rsize_t maxsize,
      const char * restrict name);
QVoid *bsearch_s(const void *key, QVoid *base, rsize_t nmemb, rsize_t size,
      int (*compar)(const void *k, const void *y, void *context),
      void *context);
errno_t qsort_s(void *base, rsize_t nmemb, rsize_t size,
      int (*compar)(const void *x, const void *y, void *context),
      void *context);
errno_t wctomb_s(int * restrict status, char * restrict s, rsize_t smax,
      wchar_t wc);
errno_t mbstowcs_s(size_t * restrict retval, wchar_t * restrict dst,
      rsize_t dstmax, const char * restrict src, rsize_t len);
errno_t wcstombs_s(size_t * restrict retval, char * restrict dst, rsize_t dstmax,
      const wchar_t * restrict src, rsize_t len);
```

B.24 _Noreturn <stdnoreturn.h>

noreturn

B.25 String handling <string.h>

size_t **NULL**
__STDC_VERSION_STRING_H__

```
void *memcpy(void * restrict s1, const void * restrict s2, size_t n);
void *memccpy(void * restrict s1, const void * restrict s2, int c, size_t n);
void *memmove(void *s1, const void *s2, size_t n);
char *strcpy(char * restrict s1, const char * restrict s2);
char *strncpy(char * restrict s1, const char * restrict s2, size_t n);
char *strdup(const char *s);
char *strndup(const char *s, size_t n);
char *strcat(char * restrict s1, const char * restrict s2);
char *strncat(char * restrict s1, const char * restrict s2, size_t n);
```

© ISO 2024 — All rights reserved

```
int memcmp(const void *s1, const void *s2, size_t n);
int strcmp(const char *s1, const char *s2);
int strcoll(const char *s1, const char *s2);
int strncmp(const char *s1, const char *s2, size_t n);
size_t strxfrm(char * restrict s1, const char * restrict s2, size_t n);
QVoid *memchr(QVoid *s, int c, size_t n);
QChar *strchr(QChar *s, int c);
size_t strcspn(const char *s1, const char *s2);
QChar *strpbrk(QChar *s1, const char *s2);
QChar *strrchr(QChar *s, int c);
size_t strspn(const char *s1, const char *s2);
QChar *strstr(QChar *s1, const char *s2);
char *strtok(char * restrict s1, const char * restrict s2);
void *memset(void *s, int c, size_t n);
void *memset_explicit(void *s, int c, size_t n);
char *strerror(int errnum);
size_t strlen(const char *s);
```

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of `<string.h>`:

errno_t **rsize_t**

```
errno_t memcpy_s(void * restrict s1, rsize_t s1max, const void * restrict s2,
      rsize_t n);
errno_t memmove_s(void *s1, rsize_t s1max, const void *s2, rsize_t n);
errno_t strcpy_s(char * restrict s1, rsize_t s1max, const char * restrict s2);
errno_t strncpy_s(char * restrict s1, rsize_t s1max, const char * restrict s2,
      rsize_t n);
errno_t strcat_s(char * restrict s1, rsize_t s1max, const char * restrict s2);
errno_t strncat_s(char * restrict s1, rsize_t s1max, const char * restrict s2,
      rsize_t n);
char *strtok_s(char * restrict s1, rsize_t * restrict s1max,
      const char * restrict s2, char ** restrict ptr);
errno_t memset_s(void *s, rsize_t smax, int c, rsize_t n)
errno_t strerror_s(char *s, rsize_t maxsize, errno_t errnum);
size_t strerrorlen_s(errno_t errnum);
size_t strnlen_s(const char *s, size_t maxsize);
```

B.26 Type-generic math `<tgmath.h>`

__STDC_VERSION_TGMATH_H__
acos
asin
atan
acosh
asinh
atanh
cos
sin
tan
cosh
sinh
tanh
exp
log
pow
sqrt
fabs
acospi
asinpi
atan2pi
atan2
atanpi
cbrt
ceil
compoundn
copysign
cospi
erfc
erf
exp10m1
exp10
exp2m1
exp2
expm1
fdim
floor
fmax
fmaximum
fmaximum_mag
fmaximum_num
fmaximum_mag_num
fma
fmin
fminimum
fminimum_mag
fminimum_num
fminimum_mag_num

© ISO 2024 — All rights reserved

`fmod`	`lrint`	`sinpi`
`frexp`	`lround`	`tanpi`
`fromfpx`	`nearbyint`	`tgamma`
`fromfp`	`nextafter`	`trunc`
`hypot`	`nextdown`	`ufromfpx`
`ilogb`	`nexttoward`	`ufromfp`
`ldexp`	`nextup`	`fadd`
`lgamma`	`pown`	`dadd`
`llogb`	`powr`	`fsub`
`llrint`	`remainder`	`dsub`
`llround`	`remquo`	`fmul`
`log10p1`	`rint`	`dmul`
`log10`	`rootn`	`fdiv`
`log1p`	`roundeven`	`ddiv`
`log2p1`	`round`	`ffma`
`log2`	`rsqrt`	`dfma`
`logb`	`scalbln`	`fsqrt`
`logp1`	`scalbn`	`dsqrt`

Only if the implementation does not define `__STDC_NO_COMPLEX__`:

`carg`	`cimag`	`conj`	`cproj`	`creal`

Only if the implementation defines `__STDC_IEC_60559_DFP__`:

`d32add`	`d64sub`	`d32div`	`d64fma`	`quantize`	`llquantexp`
`d64add`	`d32mul`	`d64div`	`d32sqrt`	`samequantum`	
`d32sub`	`d64mul`	`d32fma`	`d64sqrt`	`quantum`	

B.27 Threads `<threads.h>`

`__STDC_NO_THREADS__`	`mtx_t`	`mtx_timed`
`ONCE_FLAG_INIT`	`tss_dtor_t`	`thrd_timedout`
`TSS_DTOR_ITERATIONS`	`thrd_start_t`	`thrd_success`
`cnd_t`	`once_flag`	`thrd_busy`
`thrd_t`	`mtx_plain`	`thrd_error`
`tss_t`	`mtx_recursive`	`thrd_nomem`

```
void call_once(once_flag *flag, void (*func)(void));
int cnd_broadcast(cnd_t *cond);
void cnd_destroy(cnd_t *cond);
int cnd_init(cnd_t *cond);
int cnd_signal(cnd_t *cond);
int cnd_timedwait(cnd_t * restrict cond, mtx_t * restrict mtx,
      const struct timespec * restrict ts);
int cnd_wait(cnd_t *cond, mtx_t *mtx);
void mtx_destroy(mtx_t *mtx);
int mtx_init(mtx_t *mtx, int type);
int mtx_lock(mtx_t *mtx);
int mtx_timedlock(mtx_t * restrict mtx, const struct timespec * restrict ts);
int mtx_trylock(mtx_t *mtx);
int mtx_unlock(mtx_t *mtx);
int thrd_create(thrd_t *thr, thrd_start_t func, void *arg);
thrd_t thrd_current(void);
int thrd_detach(thrd_t thr);
int thrd_equal(thrd_t thr0, thrd_t thr1);
```

© ISO 2024 — All rights reserved

```
[[noreturn]] void thrd_exit(int res);
int thrd_join(thrd_t thr, int *res);
int thrd_sleep(const struct timespec *duration, struct timespec *remaining);
void thrd_yield(void);
int tss_create(tss_t *key, tss_dtor_t dtor);
void tss_delete(tss_t key);
void *tss_get(tss_t key);
int tss_set(tss_t key, void *val);
```

B.28 Date and time <time.h>

__STDC_VERSION_TIME_H__ **TIME_UTC** **time_t**
NULL **size_t** **struct timespec**
CLOCKS_PER_SEC **clock_t** **struct tm**

```
clock_t clock(void);
double difftime(time_t time1, time_t time0);
time_t mktime(struct tm *timeptr);
time_t timegm(struct tm *timeptr);
time_t time(time_t *timer);
int timespec_get(struct timespec *ts, int base);
int timespec_getres(struct timespec *ts, int base);
[[deprecated]] char *asctime(const struct tm *timeptr);
[[deprecated]] char *ctime(const time_t *timer);
struct tm *gmtime(const time_t *timer);
struct tm *gmtime_r(const time_t *timer, struct tm *buf);
struct tm *localtime(const time_t *timer);
struct tm *localtime_r(const time_t *timer, struct tm *buf);
size_t strftime(char * restrict s, size_t maxsize, const char * restrict format,
      const struct tm * restrict timeptr);
```

Only if supported by the implementation:

TIME_MONOTONIC **TIME_ACTIVE**

Only if threads are supported and it is supported by the implementation:

TIME_THREAD_ACTIVE

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of <time.h>:

errno_t **rsize_t**

```
errno_t asctime_s(char *s, rsize_t maxsize, const struct tm *timeptr);
errno_t ctime_s(char *s, rsize_t maxsize, const time_t *timer);
struct tm *gmtime_s(const time_t * restrict timer, struct tm * restrict result);
struct tm *localtime_s(const time_t * restrict timer, struct tm * restrict result);
```

B.29 Unicode utilities <uchar.h>

__STDC_VERSION_UCHAR_H__ **size_t** **char16_t**
mbstate_t **char8_t** **char32_t**

© ISO 2024 — All rights reserved

```
size_t mbrtoc8(char8_t * restrict pc8, const char * restrict s, size_t n,
      mbstate_t * restrict ps);
size_t c8rtomb(char * restrict s, char8_t c8, mbstate_t * restrict ps);
size_t mbrtoc16(char16_t * restrict pc16, const char * restrict s, size_t n,
      mbstate_t * restrict ps);
size_t c16rtomb(char * restrict s, char16_t c16, mbstate_t * restrict ps);
size_t mbrtoc32(char32_t * restrict pc32, const char * restrict s, size_t n,
      mbstate_t * restrict ps);
size_t c32rtomb(char * restrict s, char32_t c32, mbstate_t * restrict ps);
```

B.30 Extended multibyte/wide character utilities <wchar.h>

```
wchar_t                 struct tm               WCHAR_MIN
size_t                  __STDC_VERSION_WCHAR_H__ WEOF
mbstate_t               NULL
wint_t                  WCHAR_MAX
```

```
int fwprintf(FILE * restrict stream, const wchar_t * restrict format, ...);
int fwscanf(FILE * restrict stream, const wchar_t * restrict format, ...);
int swprintf(wchar_t * restrict s, size_t n, const wchar_t * restrict format,
      ...);
int swscanf(const wchar_t * restrict s, const wchar_t * restrict format, ...);
int vfwprintf(FILE * restrict stream, const wchar_t * restrict format,
      va_list arg);
int vfwscanf(FILE * restrict stream, const wchar_t * restrict format,
      va_list arg);
int vswprintf(wchar_t * restrict s, size_t n, const wchar_t * restrict format,
      va_list arg);
int vswscanf(const wchar_t * restrict s, const wchar_t * restrict format,
      va_list arg);
int vwprintf(const wchar_t * restrict format, va_list arg);
int vwscanf(const wchar_t * restrict format, va_list arg);
int wprintf(const wchar_t * restrict format, ...);
int wscanf(const wchar_t * restrict format, ...);
wint_t fgetwc(FILE *stream);
wchar_t *fgetws(wchar_t * restrict s, int n, FILE * restrict stream);
wint_t fputwc(wchar_t c, FILE *stream);
int fputws(const wchar_t * restrict s, FILE * restrict stream);
int fwide(FILE *stream, int mode);
wint_t getwc(FILE *stream);
wint_t getwchar(void);
wint_t putwc(wchar_t c, FILE *stream);
wint_t putwchar(wchar_t c);
wint_t ungetwc(wint_t c, FILE *stream);
double wcstod(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
float wcstof(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
long double wcstold(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
long int wcstol(const wchar_t * restrict nptr, wchar_t ** restrict endptr,
      int base);
long long int wcstoll(const wchar_t * restrict nptr, wchar_t ** restrict endptr,
      int base);
unsigned long int wcstoul(const wchar_t * restrict nptr,
      wchar_t ** restrict endptr, int base);
unsigned long long int wcstoull(const wchar_t * restrict nptr,
      wchar_t ** restrict endptr, int base);
wchar_t *wcscpy(wchar_t * restrict s1, const wchar_t * restrict s2);
wchar_t *wcsncpy(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
wchar_t *wmemcpy(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
wchar_t *wmemmove(wchar_t *s1, const wchar_t *s2, size_t n);
```

© ISO 2024 — All rights reserved

```
wchar_t *wcscat(wchar_t * restrict s1, const wchar_t * restrict s2);
wchar_t *wcsncat(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
int wcscmp(const wchar_t *s1, const wchar_t *s2);
int wcscoll(const wchar_t *s1, const wchar_t *s2);
int wcsncmp(const wchar_t *s1, const wchar_t *s2, size_t n);
size_t wcsxfrm(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
int wmemcmp(const wchar_t *s1, const wchar_t *s2, size_t n);
QWchar_t *wcschr(QWchar_t *s, wchar_t c);
size_t wcscspn(const wchar_t *s1, const wchar_t *s2);
QWchar_t *wcspbrk(QWchar_t *s1, const wchar_t *s2);
QWchar_t *wcsrchr(QWchar_t *s, wchar_t c);
size_t wcsspn(const wchar_t *s1, const wchar_t *s2);
QWchar_t *wcsstr(QWchar_t *s1, const wchar_t *s2);
wchar_t *wcstok(wchar_t * restrict s1, const wchar_t * restrict s2,
      wchar_t ** restrict ptr);
QWchar_t *wmemchr(QWchar_t *s, wchar_t c, size_t n);
size_t wcslen(const wchar_t *s);
wchar_t *wmemset(wchar_t *s, wchar_t c, size_t n);
size_t wcsftime(wchar_t * restrict s, size_t maxsize,
      const wchar_t * restrict format, const struct tm * restrict timeptr);
wint_t btowc(int c);
int wctob(wint_t c);
int mbsinit(const mbstate_t *ps);
size_t mbrlen(const char * restrict s, size_t n, mbstate_t * restrict ps);
size_t mbrtowc(wchar_t * restrict pwc, const char * restrict s, size_t n,
      mbstate_t * restrict ps);
size_t wcrtomb(char * restrict s, wchar_t wc, mbstate_t * restrict ps);
size_t mbsrtowcs(wchar_t * restrict dst, const char ** restrict src, size_t len,
      mbstate_t * restrict ps);
size_t wcsrtombs(char * restrict dst, const wchar_t ** restrict src, size_t len,
      mbstate_t * restrict ps);
```

Only if the implementation defines **__STDC_IEC_60559_DFP__**:

```
_Decimal32 wcstod32(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
_Decimal64 wcstod64(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
_Decimal128 wcstod128(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
```

Only if the implementation defines **__STDC_LIB_EXT1__** and additionally the user code defines **__STDC_WANT_LIB_EXT1__** before any inclusion of <wchar.h>:

errno_t **rsize_t**

```
int fwprintf_s(FILE * restrict stream, const wchar_t * restrict format, ...);
int fwscanf_s(FILE * restrict stream, const wchar_t * restrict format, ...);
int snwprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
      ...);
int swprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
      ...);
int swscanf_s(const wchar_t * restrict s, const wchar_t * restrict format, ...);
int vfwprintf_s(FILE * restrict stream, const wchar_t * restrict format,
      va_list arg);
int vfwscanf_s(FILE * restrict stream, const wchar_t * restrict format,
      va_list arg);
int vsnwprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
      va_list arg);
int vswprintf_s(wchar_t * restrict s, rsize_t n, const wchar_t * restrict format,
      va_list arg);
int vswscanf_s(const wchar_t * restrict s, const wchar_t * restrict format,
      va_list arg);
```


© ISO 2024 — All rights reserved

```
int vwprintf_s(const wchar_t * restrict format, va_list arg);
int vwscanf_s(const wchar_t * restrict format, va_list arg);
int wprintf_s(const wchar_t * restrict format, ...);
int wscanf_s(const wchar_t * restrict format, ...);
errno_t wcscpy_s(wchar_t * restrict s1, rsize_t s1max,
      const wchar_t * restrict s2);
errno_t wcsncpy_s(wchar_t * restrict s1, rsize_t s1max,
      const wchar_t * restrict s2, rsize_t n);
errno_t wmemcpy_s(wchar_t * restrict s1, rsize_t s1max,
      const wchar_t * restrict s2, rsize_t n);
errno_t wmemmove_s(wchar_t *s1, rsize_t s1max, const wchar_t *s2, rsize_t n);
errno_t wcscat_s(wchar_t * restrict s1, rsize_t s1max,
      const wchar_t * restrict s2);
errno_t wcsncat_s(wchar_t * restrict s1, rsize_t s1max,
      const wchar_t * restrict s2, rsize_t n);
wchar_t *wcstok_s(wchar_t * restrict s1, rsize_t * restrict s1max,
      const wchar_t * restrict s2, wchar_t ** restrict ptr);
size_t wcsnlen_s(const wchar_t *s, size_t maxsize);
errno_t wcrtomb_s(size_t * restrict retval, char * restrict s, rsize_t smax,
      wchar_t wc, mbstate_t * restrict ps);
errno_t mbsrtowcs_s(size_t * restrict retval, wchar_t * restrict dst,
      rsize_t dstmax, const char ** restrict src, rsize_t len,
      mbstate_t * restrict ps);
errno_t wcsrtombs_s(size_t * restrict retval, char * restrict dst,
      rsize_t dstmax, const wchar_t ** restrict src, rsize_t len,
      mbstate_t * restrict ps);
```

B.31 Wide character classification and mapping utilities <wctype.h>

wint_t **wctrans_t** **wctype_t** **WEOF**

```
int iswalnum(wint_t wc);
int iswalpha(wint_t wc);
int iswblank(wint_t wc);
int iswcntrl(wint_t wc);
int iswdigit(wint_t wc);
int iswgraph(wint_t wc);
int iswlower(wint_t wc);
int iswprint(wint_t wc);
int iswpunct(wint_t wc);
int iswspace(wint_t wc);
int iswupper(wint_t wc);
int iswxdigit(wint_t wc);
int iswctype(wint_t wc, wctype_t desc);
wctype_t wctype(const char *property);
wint_t towlower(wint_t wc);
wint_t towupper(wint_t wc);
wint_t towctrans(wint_t wc, wctrans_t desc);
wctrans_t wctrans(const char *property);
```

© ISO 2024 — All rights reserved